

RS002 Week 8 — English Worksheet

Topic: Validation Everywhere + Logging Bad Input · **Course:** Pokédex App · **Time:** about 45 minutes

This week is about thinking through the validation pattern for **every** input in your app — name, region, search — and how each bad attempt gets **logged** into a list for a session report. You also meet your first **function** here: a reusable `safe_input`. You will read code, trace it, and explain the code you wrote in your live lesson — no typing in the app today.

Words to know: **function**, `safe_input`, **log**, **append**, **return**, **report**

1 · Recall

From memory:

Write an `if` that warns and falls back to `"피카츄"` when `choice` is `not in` the list `pokedex`.

2 · Predict 🧠

Read each block. Write what you think will happen — just from reading, without running it.

```
invalid_attempts = []
invalid_attempts.append("리자몽")
invalid_attempts.append("타입null")
print(len(invalid_attempts))
print(invalid_attempts)
```

What two lines print?


```
invalid_attempts = ["리자몽", "타입null"]
print(f"잘못 입력: {' , '.join(invalid_attempts)}")
```

`' , '.join(...)` glues list items with a comma between them. What prints?


```
def safe_input(prompt, default=""):
    answer = input(prompt).strip()
    if answer == "":
        answer = default
    return answer

name = safe_input("이름: ", "이름 모를 트레이너")
print(name)
```

The user presses Enter. What gets printed? Trace the path through the function.

3 · Spot the Bug 🐛

Read what each block is **supposed** to do, fix it on paper, then explain the fix.

Bug A — Each bad answer should be **added** to the log. Right now it replaces the whole log every time, so only the last one survives.

```
invalid_attempts = []
invalid_attempts = "리자몽"
invalid_attempts = "타입null"
print(invalid_attempts)
```

Hint: use `.append(...)` to add to a list.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — This function should **return** the cleaned answer so the caller can use it. Right now it prints inside and returns nothing.

```
def safe_input(prompt, default=""):
    answer = input(prompt).strip()
    if answer == "":
        answer = default
    print(answer)

name = safe_input("이름: ", "트레이너")
print(f"환영합니다, {name}!")
```

Hint: swap the `print` for a `return`.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — The report should only list bad attempts **if there were any**. Right now it prints an empty list and a confusing message even when everything was fine.

```
invalid_attempts = []  
print(f"잘못 입력한 내용: {'', '.join(invalid_attempts)}")
```

Hint: wrap the report in `if invalid_attempts:` .

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Explain the Code

Here is a working `safe_input` with a log and a session report. Read it carefully and answer the questions.

```
invalid_attempts = []

def safe_input(prompt, valid_list=None, default=""):
    answer = input(prompt).strip()

    if valid_list and answer not in valid_list:
        invalid_attempts.append(answer)
        print(f" ⚠️ '{answer}' 은(는) 인식 못해요. 다시 시도하세요.")
        answer = input(prompt).strip()
        if valid_list and answer not in valid_list:
            invalid_attempts.append(answer)
            print(f" 기본값 ({default})으로 진행합니다.")
            answer = default

    if answer == "" and default:
        answer = default

    return answer

pokedex = ["피카츄", "이상해씨", "파이리"]
choice = safe_input("포켓몬 이름: ", pokedex, "피카츄")
print(f"선택: {choice}")

print(f"\n=== 세션 통계 ===")
print(f"잘못된 입력 횟수: {len(invalid_attempts)}")
if invalid_attempts:
    print(f"잘못 입력한 내용: {' '.join(invalid_attempts)}")
```

What does the line `invalid_attempts.append(answer)` do, and when does it run?

Why does `safe_input` end with `return answer` instead of `print(answer)` ?

What are the three things you pass into `safe_input` on the `choice = ...` line, and what is each one for?

Walk through what happens if the user types a bad name twice in a row. Which lines run?

Why is the report wrapped in `if invalid_attempts:` ?

5 · Explain Your Lesson Code 🎥

In today's live lesson you wrote your own `safe_input` and applied it to your Pokédex. Explain **the code you wrote** in a short phone video. You may show it running with some bad input on purpose. Try to use: **function, safe_input, log, append, report**.

First, I wrote a function called `safe_input` that ...

I logged bad attempts by ...

I used the same function for ... different inputs.

My session report showed ...

Write what you will say in your video. Plan it here before you record.

Submit 

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.